RSCL — Algorithm 5: Formation Slot Assignment with Sampled Straight-Line Conflict Checking and Translated Placement (assign_shifted)

Reconfigurable Space Computing Lab (RSCL), California State Polytechnic University, Pomona (Cal Poly Pomona) · Director: Prof. Mohamed El-Hadedy

Date: 2026-11-12 · Document type: formal algorithm specification of the implemented code (HU-MA'AT drone-fleet simulation demo)

Source files described (content hash and state at document build time):

`hu-telepresence-gazebo/festival/formation_planner.py`	`sha256 b3d8e9fdf6a8`	2026-11-12 02:01 UTC	clean @ f50621f
`hu-telepresence-gazebo/festival/control_server_hu.py`	`sha256 662fd7191f00`	2026-11-12 02:11 UTC	clean @ e14ef3f
`hu-telepresence-gazebo/festival/formation_catalog.json`	`sha256 c49a61a92a27`	2026-11-12 01:32 UTC	clean @ 05172be
`hu-telepresence-gazebo/acceptance/formation_catalog_test.py`	`sha256 b94f88fed182`	2026-11-12 00:56 UTC	clean @ 05172be
`hu-telepresence-gazebo/acceptance/height_profile.py`	`sha256 ad6998e5b6eb`	2026-11-12 01:19 UTC	clean @ 05172be

1. Purpose and context

A formation command gives 2–5 airborne aircraft new horizontal slots at a common altitude. The planner must choose which aircraft takes which slot so that the simultaneous transition does not bring two aircraft too close, and must keep clear of aircraft that are not part of the formation (for example a faulted aircraft hovering mid-arena). It is a small, bounded, non-learning planner: exhaustive search over assignments with a sampled geometric check. It produces setpoints, not trajectories; the low-level controller and the proximity watchdog (Algorithm 7) remain responsible at run time.

2. Inputs, outputs, data structures

Item	Definition
shape, n	Shape key, $2 \le n \le 5$. Current revision: 106 keys reachable from 239 phrases (Algorithm 3), of which 104 are distinct shapes and 2 are aliases with identical geometry (pentagon→circle, relay_chain→line). Geometry sources: closed-form or fixed tables in `slots`, 58 fixed 5-point layouts (`EXTRA_SHAPES`), 18 linear transforms of base shapes (`DERIVED_SHAPES`). `HEIGHT_PROFILES` (11 3D profiles) are applied after this algorithm, see below. Live status per shape is kept in `festival/formation_catalog.json` (snapshot 2026-11-12 01:32 UTC: 105 entries VALIDATED, i.e. 103 distinct geometries plus the 2 aliases, and 1 EXPERIMENTAL, `pentomino_w`; see Figure 5.2).
Participants P	Ordered names; current positions p_i = (x,y,z) from odometry.
Obstacles O	Positions of all non-participating aircraft, treated as stationary.
Output	`assign`: list of slot points in participant order; `assign_shifted`: (slots, (dx,dy)). The supervisor sets target (x, y, 1.90 m) and mode FORMATION.

3D height profiles. `formation_planner.HEIGHT_PROFILES` holds 11 named profiles (pyramid_3d, staircase, tower, helix, wave, saddle, dome, ramp, v_stack, diamond_peak, pyramid_peak), each a base 2D shape and a vertical offset per slot. `acceptance/height_profile.py` (used by the show, Algorithm 16) runs only after the 2D formation of this algorithm has been flown: it matches each of the five aircraft to its slot of `slots(base, 5)` by nearest horizontal distance, rounds the offset to 10 cm steps and clamps the resulting altitude to 1.1–2.7 m. The output is purely

vertical moves, issued one aircraft at a time through HU and MA'AT, so the horizontal slot spacing (≥ 0.88 m) and the assignment below are unchanged.

3. Algorithm

Algorithm 5.1 SLOTS (geometry)

Source: festival/formation_planner.py: slots

```
function SLOTS(shape, n)
  require 2 ≤ n ≤ 5
  S ← shape template (closed-form or fixed table truncated to n points)   // e.g. line: (1.15(i−(n−1)/2), 0)
  if shape is DERIVED (base, [a b; c d]) then S ← [ (a·x + b·y, c·x + d·y) for (x,y) in SLOTS(base, 5) ][:n]
  c ← centroid(S);  S ← S − c                                              // centre on the arena origin
  if any |x| > 3.0 or |y| > 3.0 then raise "exceeds arena"
  if any pair of slots closer than 0.88 m then raise "separation too small"
  return S
```

Algorithm 5.2 ASSIGN (permutation search with sampled conflict check)

Source: festival/formation_planner.py: assign

```
function ASSIGN(P, drones, S, O)
  cur ← [ (x,y,z) of p ∈ P ];  obs ← [ (x,y,z) of o ∈ O ];  best ← None
  for each permutation π of S do                            // n! ≤ 120 candidates
    g_i ← (π_i.x, π_i.y, 1.90)
    if ∃ i, o: ‖g_i.xy − o.xy‖ < 0.88 then continue      // slot must also be a safe parking spot (vertical landing)
    for k = 0..40 do                                        // f = k/40, all movers advance together
      q_i ← cur_i + (g_i − cur_i)·k/40
      if ∃ i<j: ‖q_i − q_j‖ < 0.88 then reject π
      if ∃ i, o: ‖q_i − o‖ < 0.88 then reject π
    cost ← Σ_i ‖g_i − cur_i‖²
    if best = None or cost < best.cost then best ← (cost, π)    // strict <: first optimum kept
  if best = None then raise "No separation-safe straight-line assignment"
  return best.π
```

Algorithm 5.3 ASSIGN-SHIFTED (translated placement around non-participants)

Source: festival/formation_planner.py: assign_shifted; festival/control_server_hu.py: Fleet.formation

```
function ASSIGN-SHIFTED(P, drones, S, O, step = 0.35, max_shift = 1.75)
  D ← [(0,0)] ++ [ (r cos(aπ/8), r sin(aπ/8)) : r = k·step, k = 1..5, a = 0..15 ]   // 81 offsets, nearest ring first
  for (dx,dy) in D do
    S' ← S + (dx,dy)
    if any slot of S' has |x| > 3.0 or |y| > 3.0 then continue
    try return (ASSIGN(P, drones, S', O), (dx,dy))
    except no-assignment: remember error
  raise last error
procedure FORMATION(shape, targets)                         // supervisor, under the fleet lock
  P ← fleet-ordered targets;  require |P| ≥ 2
  require all five aircraft have fresh odometry;  require no participant faulted
  require every participant in {HOVER, MOVE, FORMATION, TURN, SPIN} and z ≥ 0.70
  S ← SLOTS(shape, |P|);  O ← fleet \ P
  if O ≠ ∅ then (slots, off) ← ASSIGN-SHIFTED(P, dr, S, O) else slots ← ASSIGN(P, dr, S, ∅)
  for each participant: target ← (slot.x, slot.y, 1.90); mode ← FORMATION
```

4. Flowchart and state diagram

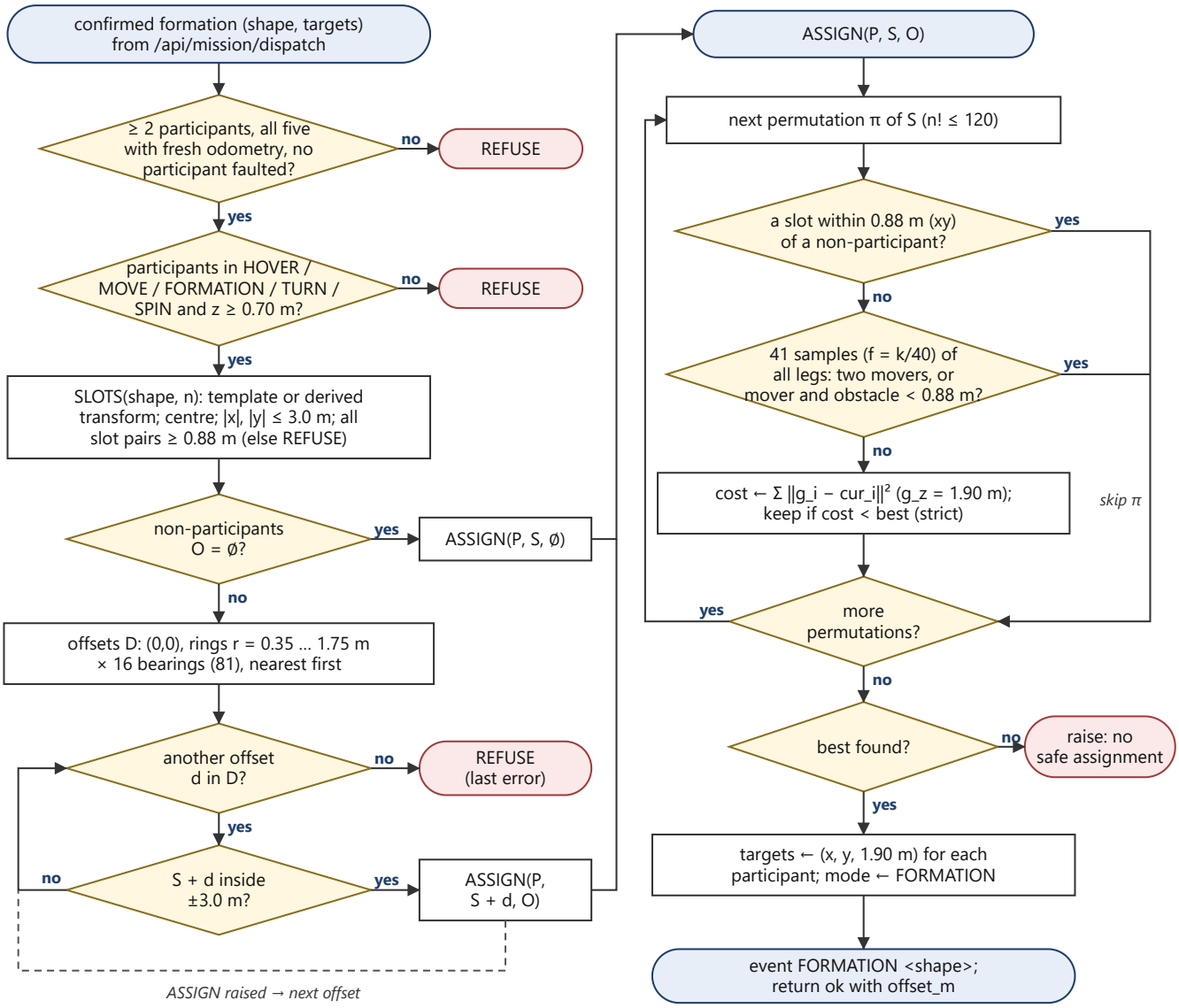


Figure 5.1. Flowchart of `Fleet.formation`, `assign_shifted` and `assign`. The whole computation runs under the fleet lock. With non-participants present, an ASSIGN failure is caught and the next translation is tried; geometry is never scaled or squeezed. The catalogue status of the shape is not consulted.

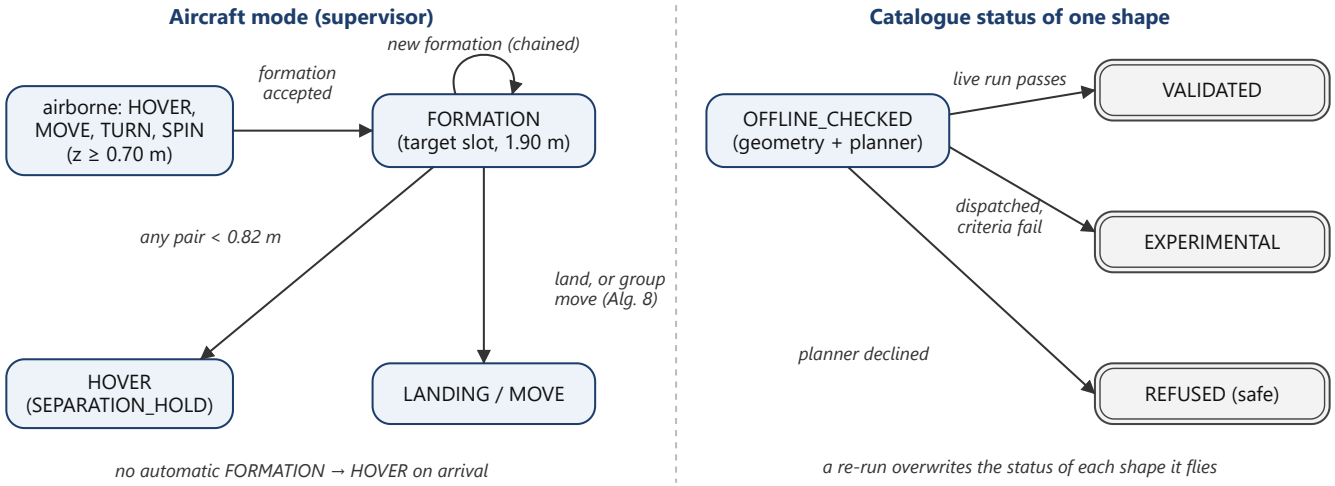


Figure 5.2. Left: per-aircraft modes around a formation command (`Fleet.formation`, `Fleet.loop`); the 0.82 m proximity hold is Algorithm 7. Right: status of a catalogue entry as written by `acceptance/formation_catalog_test.py` (VALIDATED requires dispatch, settle ≤ 30 s, RMS slot error ≤ 0.10 m, no proximity hold and observed separation ≥ 0.82 m). Only the mission coordinator (Algorithm 11) reads this status; the live command path of this algorithm does not.

5. Parameters

Parameter	Value	Note
MIN_SEP	0.88 m	slot spacing, path samples, obstacle clearance (module docstring says 0.9 m; code uses 0.88)
BOUND	3.0 m	|x|, |y| of every slot (supervisor move bound is 3.35 m)
ALT	1.90 m	formation altitude for all slots
Path samples	41 (f = 0, 1/40, ..., 1)	3D Euclidean checks
Shift grid	step 0.35 m, max 1.75 m, 16 bearings; 81 candidates	nearest offset first
Participant preconditions	z ≥ 0.70 m, airborne mode, not faulted; all 5 fresh	Fleet.formation

6. Complexity

ASSIGN: n! permutations × 41 samples × (C(n,2) + n·|O|) distance evaluations; for n = 5, |O| = 0: $120 \times 41 \times 10 \approx 4.9 \cdot 10^4$. ASSIGN-SHIFTED multiplies this by at most 81 (n = 4, |O| = 1: $24 \times 41 \times 10 \times 81 \approx 8 \cdot 10^5$). Factorial in n; acceptable only because n ≤ 5. The plan is computed while the supervisor holds the fleet lock (the 10 Hz control loop waits for it); the dry-run path of Algorithm 10 runs it outside the lock.

7. Properties

Guaranteed (on the model used)

- Every returned slot lies inside ±3.0 m and every pair of slots is ≥ 0.88 m apart.
- If movers travel on straight lines at speeds proportional to their leg lengths (all arriving together), no sampled instant has two movers, or a mover and a stationary aircraft, closer than 0.88 m in 3D.
- No slot is horizontally within 0.88 m of a non-participant, so participants can later land vertically next to it.
- The returned assignment minimises the sum of squared leg lengths among the feasible permutations (exact, exhaustive). Deterministic for a given input.
- Formation geometry is never squeezed to avoid obstacles: only translation is tried; failure is a refusal.

Not guaranteed

- **The motion model is not the controller.** The supervisor (Algorithm 9) clamps each horizontal velocity axis to 0.62 m/s and the vertical to 0.45 m/s with a PD law, so real paths are neither straight nor synchronised; the sampled check is a heuristic, not a proof.
- Sampling at 1/40 of the path can miss a closer approach between samples (for legs up to ~5 m, a sample spacing of ~0.13 m).
- Non-participants are assumed stationary for the duration.
- Minimum-cost is in squared distance, not time; longest-leg (makespan) is not minimised.
- Assignment is not stable across small position changes (ties and near-ties may flip slots).
- **The supervisor does not consult the catalogue status.** Any shape reachable by phrase (Algorithm 3) is flown if it passes the checks above, whether its catalogue entry is VALIDATED, OFFLINE_CHECKED (never flown), EXPERIMENTAL (flown, criteria failed) or REFUSED (planner declined in the recorded run). Only the mission coordinator (Algorithm 11) restricts itself to VALIDATED shapes. In the snapshot, 1 shape is not VALIDATED: pentomino_w.

8. Testing and measured results

- Offline: `acceptance/tests/test_missions.py` (`test_formation_controller`, `test_collision_checker`, `test_grounded_no_formation`, `test_fault_blocks_formation`, `test_shape_catalogue`); `acceptance/formation_geometry_check.py` (slot spacing and extent of every shape).
- Live, `acceptance/fleet_test.py` (run `run_20261111T052914Z`): line, V, circle, grid, diamond all PASS, settle 2.82–4.25 s, RMS slot error 0.051–0.083 m, minimum observed separation 0.998 m (Thoth–Anubis) over 246 samples.
- Live catalogue, `acceptance/formation_catalog_test.py`, summarised in `festival/formation_catalog.json` (snapshot 2026-11-12 01:32 UTC; evidence files: `formation_catalog_20261111T053652Z.json` (20); `formation_catalog_20261111T053814Z.json` (1); `formation_catalog_20261112T011849Z.json` (42); `formation_catalog_20261112T013243Z.json` (43)). Three live validation passes were run on 2026-11-12: `acceptance/results/catalog_new/formation_catalog_20261112T011849Z` (44 VALIDATED, 1 EXPERIMENTAL, 47 REFUSED (safe)); `catalog_new2/formation_catalog_20261112T012039Z` (a pass with a frozen fleet: 1 EXPERIMENTAL, 56 REFUSED (safe); treated as an artefact, its results are not counted and no catalogue entry cites it); `catalog_new3/formation_catalog_20261112T013243Z` (65/65 VALIDATED; the earlier file `...012335Z` in the same folder is empty). The only EXPERIMENTAL entry, `pentomino_w` (from the first pass), recorded observed minimum separation 0.633 m against 1.25 m planned. Across the 105 VALIDATED entries the recorded RMS slot error is 0.008–0.027 m and the observed minimum separation 0.906 m (shape l) to 2.333 m (shape big_x). VALIDATED requires 0 proximity-watchdog holds and observed separation ≥ 0.82 m; values between 0.82 m and 1.0 m are therefore possible for VALIDATED shapes. The catalogue is rewritten by each run, so these figures describe the snapshot, not a fixed result.
- Offline: `acceptance/tests/test_formation_catalog_offline.py` (every catalogue shape builds five safe slots; shapes work for 2–4 aircraft; catalogue status is honest; aliases target buildable shapes). 80 VALIDATED entries still carry the note "Offline geometry + planner check only; not flown in Gazebo yet", which is out of date relative to their status and evidence fields.
- Live, `acceptance/fault_test.py`: four-aircraft V re-planned around faulted Anubis, settle 4.25 s, minimum separation to the faulted aircraft 0.929 m (PASS).

9. Comparison with alternatives and related work

Formation reconfiguration combines a goal-assignment problem with multi-robot motion planning. Five established methods are compared.

- **Hungarian method** [1]: optimal linear-sum assignment in polynomial time ($O(n^3)$ in standard implementations).
- **Auction algorithm** [2]: an ε-optimal, parallelisable assignment method in which agents bid for goals; suited to decentralised implementation.
- **CAPT** [3]: for interchangeable robots in free space, assigning goals by minimum sum of squared distances and moving all robots on synchronised straight lines yields collision-free trajectories when start and goal positions are sufficiently separated (the paper states the condition in terms of the robot radius R; as we read it, more than $2\sqrt{2}\cdot R$).
- **Conflict-Based Search (CBS)** [4]: optimal multi-agent path finding on a graph; a high-level search over conflicts adds constraints to single-agent searches, so agents may wait or detour.
- **ORCA** [5]: decentralised, reactive collision avoidance; each agent picks a velocity outside the reciprocal velocity obstacles of its neighbours at every control step.

Method	Assignment	Collision handling	Scales to	This planner, by comparison
Hungarian [1]	optimal, linear sum	none	hundreds+ ($O(n^3)$)	same optimum for its cost, found by n! enumeration; feasible only for $n \le 5$
Auction [2]	ε-optimal, distributed	none	large, parallel	centralised, under one lock
CAPT [3]	min Σ squared distance	guaranteed under a separation condition, synchronised straight lines	large teams	same objective and motion model, but the guaranteed slot spacing (0.88 m) is below the condition for 0.88 m clearance (about 1.24 m), so separation is sampled instead of guaranteed
CBS [4]	given (or combined in variants)	optimal conflict-free paths on a graph; waits and detours	tens to hundreds of agents	no waits or detours; only refusal or rigid translation
ORCA [5]	none	reactive, per step, decentralised	thousands of agents	runtime protection is a stop (0.82 m hold, Alg. 7), not avoidance

Where this planner is weaker. Its run time is factorial, its conflict test is sampled rather than exact, it assumes a motion model (simultaneous straight lines) that the PD controller does not follow, it never sequences or delays movers, and it treats non-participants as static. [3] gives a guarantee for the same cost function under conditions this catalogue does not meet; [4] would find conflict-free plans in cases this planner refuses. **Where it is adequate.** For $n \le 5$ the enumeration is exact and fast, the result is deterministic and auditable, and every candidate is checked against the same bounds the supervisor enforces; unsafe cases end in a refusal, never in a squeezed formation.

10. Relation to other algorithms

Invoked for formation proposals from Algorithm 3 and by Algorithm 11 (via Algorithm 10); reused offline by Algorithms 14 and 18 (`assign`, `slots`, `ALT`). Runtime protection is Algorithm 7; arrival is tracked by Algorithm 9 (FORMATION mode has no automatic HOVER transition).

11. References

1. H. W. Kuhn, "The Hungarian method for the assignment problem," *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955. https://doi.org/10.1002/nav.3800020109
2. D. P. Bertsekas, "The auction algorithm: A distributed relaxation method for the assignment problem," *Annals of Operations Research*, 14:105–123, 1988. https://doi.org/10.1007/BF02186476
3. M. Turpin, N. Michael and V. Kumar, "CAPT: Concurrent assignment and planning of trajectories for multiple robots," *International Journal of Robotics Research*, 33(1):98–112, 2014. https://doi.org/10.1177/0278364913515307

4. G. Sharon, R. Stern, A. Felner and N. R. Sturtevant, "Conflict-based search for optimal multi-agent pathfinding," *Artificial Intelligence*, 219:40–66, 2015. https://doi.org/10.1016/j.artint.2014.11.006
5. J. van den Berg, S. J. Guy, M. Lin and D. Manocha, "Reciprocal n-Body Collision Avoidance," in *Robotics Research*, Springer Tracts in Advanced Robotics 70, 2011, pp. 3–19. https://doi.org/10.1007/978-3-642-19457-3_1

This specification describes the behaviour of the source revision identified above. Line-level references use `file:function`. Where the code and its own comments disagree, the code is described. Simulation-only system: no statement here is a claim about flight hardware.